Designed by Remus Teodorescu 2026

THE ARC

Why This Half-Lecture Exists

AN HONEST ACCOUNT OF WHAT IT BUYS YOU

Last time was the language. This time is the object every later lecture is actually made of. A neural network is a stack of arrays; a training batch is an array; the output of a physics-informed model is an array, and so is the residual you compare it against.

So the vocabulary matters more here than it looks. If you cannot say what shape something is, you cannot say what a layer did to it, and from lecture four onwards that is most of the debugging.

I want to be specific about the claim on the right, because it is the reason this material is taught at all rather than assumed. Across cohorts, the great majority of the bugs that cost a student an afternoon in the second half of this course are broadcasting misunderstandings. Not gradient problems, not architecture — shapes.

Twenty minutes here. An entire exercise session in week seven. That is the trade, and it is why broadcasting gets four slides and a drill notebook.

WHAT TODAY IS FOR

arrays	the object, and why not a list
dtype, shape	the two things to print
indexing	and the fact that a slice is a view
broadcasting	the rule, and its two traps
vectorisation	measured, not asserted
figures	axes, units, log scales

THE CLAIM THIS DECK RESTS ON

Every shape bug you hit from lecture seven onwards traces back to broadcasting. That is not a figure of speech — it is what the exercise sessions actually look like, every year.

An Array Is Not a List

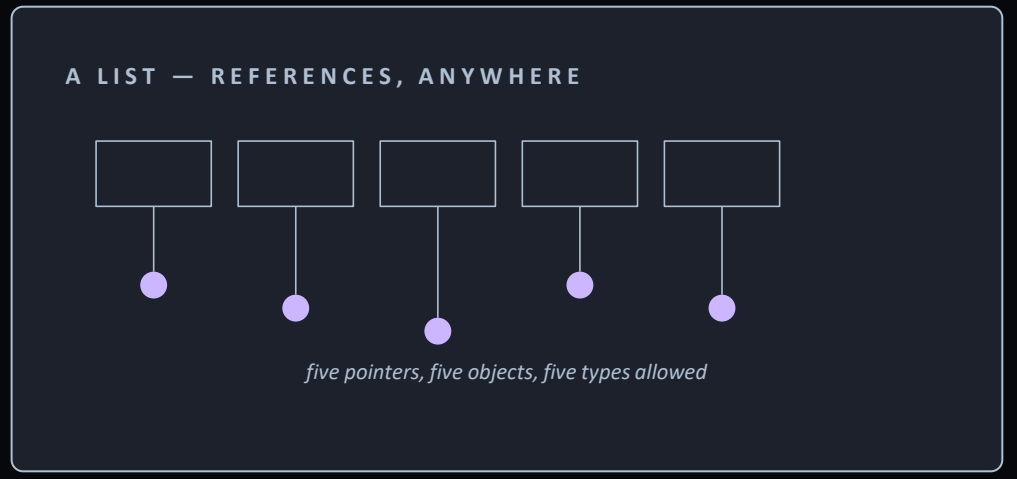
ONE BLOCK OF MEMORY, ONE TYPE, ONE SHAPE

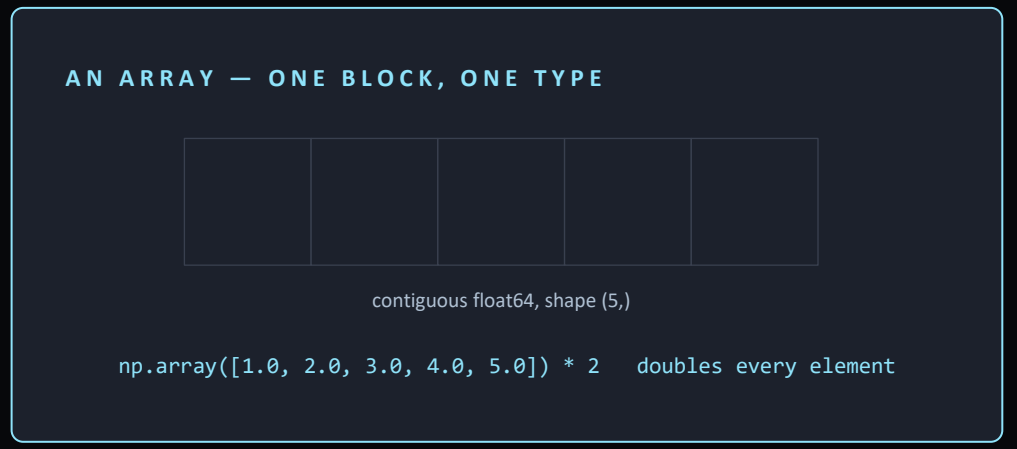
A Python list holds references to objects that can live anywhere in memory and can be of any type. A NumPy array is a single contiguous block of memory holding elements that are all the same type, with a shape describing how to read it.

That difference has three consequences, and all three matter.

Arithmetic on an array is elementwise and happens in compiled code. Multiplying a list by two repeats the list; multiplying an array by two doubles every element. That is not a subtlety, it is a different operation with the same symbol.

The memory is contiguous, so the machine can walk it without chasing pointers, which is where the speed comes from. And the type is fixed, so an array of integers stays an array of integers and will truncate a float you assign into it without warning.





dtype: the Element Type, and Its Traps

FIXED WHEN THE ARRAY IS MADE, AND ENFORCED AFTERWARDS

Every array has a dtype — the type of its elements. Build one from whole numbers and you get int64; build one from anything with a decimal point and you get float64, which is what you want for essentially all of this course.

Two traps, both of which produce wrong numbers rather than errors.

The first: assigning a float into an integer array truncates it silently. Two point seven becomes two, no warning, and the array is still an integer array afterwards.

The second becomes important in L2.2. NumPy's default is float64 and PyTorch's default is float32, so a NumPy array handed to torch without a dtype argument produces a dtype mismatch — sometimes an error, sometimes a silent upcast that quietly doubles your memory. Say the dtype explicitly at every boundary between the two libraries.

THE FOUR YOU WILL SEE

float64	NumPy's default — use it here
float32	torch's default — L2.2 onwards
int64	counts and indices only
bool	masks, from slide 7

```
counts = np.array([1, 2, 3])
counts.dtype              int64
counts[0] = 2.7           # truncates
counts[0]                 2

x = np.array([1, 2, 3], dtype=np.float64)
```

Say the dtype when it matters. It matters at every torch boundary.

THE OBJECT

shape: the Habit This Lecture Installs

PRINT IT. NOT SOMETIMES — EVERY TIME.

The shape is a tuple giving the length of each axis. Its length is the number of dimensions, which NumPy calls ndim, and the product of its entries is the number of elements.

Read a shape right to left, as a nesting. The tuple six comma two hundred and forty means six rows of two hundred and forty, so axis nought indexes gauges and axis one indexes samples. Keeping that reading straight is most of what it takes to get an array expression right first time.

Now the habit, which is the actual content of this slide. Every time you build an array, reshape one, or get one back from a function, print its shape on the very next line. It costs one line and about a second, and it turns a class of bug that takes twenty minutes into one that takes ten seconds.

I am aware this sounds like the sort of advice nobody follows. The exercise notebooks make you do it anyway, and by Ex04 most people are doing it without being asked, because by then they have been bitten.

ONE ARRAY, READ CORRECTLY

shape (6, 240) — drawn small

`axis 0 → gauges` `axis 1 → samples`

THE FOUR CONSTRUCTORS

```
np.linspace(0.0, 2.0, 41)   # endpoint included
np.arange(0, 6)             # endpoint excluded
np.zeros((2, 5))            # a shape tuple
np.random.normal(size=(3, 4))
```

One Index per Axis

AND THE DIFFERENCE THAT SEEDS THE BUG ON SLIDE 11

Index a two-dimensional array with one index per axis inside a single pair of brackets. Write a comma one two, not a bracket one bracket bracket two. The second form works, but it builds an intermediate array and it does not generalise to slices.

Here is the sentence to remember. Indexing with an integer removes an axis. Slicing with a colon keeps it.

So the first row of a six by two hundred and forty array, taken with an integer, has shape two hundred and forty — one axis. Taken as a slice of length one, it has shape one comma two hundred and forty — two axes, the same numbers.

Those two objects contain identical values and behave completely differently the moment you combine them with anything. That difference is the seed of the failure on slide eleven, and it is worth being able to see it before it costs you anything.

THE SAME NUMBERS, TWO SHAPES

`a[0, :]` → shape `(8,)`

`a[0:1, :]` → shape `(1, 8)`

```
a[1, 2]      one element, shape ()
a[:, 0]      first column, shape (4,)
a[0, :]      first row, shape (8,)
a[0:1, :]    first row, shape (1, 8)
a[1][2]      works, and do not write it
```

An integer removes an axis. A colon keeps it. Remember that one.

A Slice Is a View, Not a Copy

THE SAME LESSON AS L1.1 SLIDE 9, ON A BIGGER OBJECT

A basic slice shares memory with the array it came from. Write into the slice and the parent changes, because there is one block of numbers and two ways of looking at it.

This is what makes NumPy fast — taking a slice moves no data at all — and it is a genuine source of bugs when you assumed you had an independent working copy. If you want one, say copy.

Two forms do copy, and you should know which. A boolean mask — the array greater than three — selects the elements where a condition holds and returns a new flat array. Fancy indexing with a list of positions also copies, and returns them in the order you asked for.

Boolean masking is how you should express the points on the boundary, or the samples above the threshold. A loop with an if inside does the same job an order of magnitude more slowly and reads worse.

ONE BLOCK, TWO VIEWS

`b = a[2:5]` · `b[0] = 99.0` changes `a[2]`

THE TWO THAT COPY

```
a[a > 3.0]     boolean mask — flat, a copy
a[[0, 2, 5]]   fancy index — a copy
a[2:5].copy()  ask, and you get one

mask = (x > 0.0) & (x < 1.0)   # & not and
```

The Problem It Solves

TWO EVERYDAY OPERATIONS THAT SHOULD NOT NEED A LOOP

You have temperatures at one hundred positions and you want to subtract a single reference temperature. You have six strain gauges by two hundred and forty samples and you want to subtract a different zero offset from each gauge — six numbers, one per row.

Neither of those should require a loop, and neither should require you to build a full-size array of repeated values just to make the shapes match.

Broadcasting is the rule that lets an array of one shape take part in an operation with an array of another, by treating an axis of length one as though it were repeated. Nothing is copied: NumPy walks the memory with a stride of zero along the stretched axis, which is why it is fast as well as convenient.

So it is a genuinely good feature, and I want that on the record before I spend two slides on how it hurts people. The trouble is not that the rule is complicated. The trouble is that it succeeds in cases where you wanted it to fail.

SIX GAUGES, ONE OFFSET EACH

One number per row, applied across all two hundred and forty samples, with nothing repeated in memory.

The Rule, Stated Once

Two shapes are compatible if, aligning them from the right, every pair of lengths is either equal or one of them is 1. Missing axes on the left of the shorter shape count as 1.

THERE IS NO SECOND CLAUSE

That is the whole rule. It is one sentence and it has two halves — align from the right, then check each pair. Everything else in this lecture, and every shape error you will ever read, is a consequence of it.

Aligning from the right is the half people forget. Shapes are compared by their last axis first, so a one-dimensional array always lines up with the last axis of whatever it meets, whatever you intended.

The result shape takes the larger of each pair. An axis of length one is stretched to match; an axis of any other length must already match exactly.

Write the two shapes out, one above the other, right-aligned, and check the columns. That is what the table on the right does, and it is what the broadcast_table function in the exercise module prints for you.

THE PROCEDURE, AS A PICTURE

(6,240)	6	240
(240,)	·	240
result	**6**	**240**

240 against 240 — equal. Then 6 against a missing axis, which counts as 1 and is stretched. The one-dimensional array is applied to every row.

Equal, or one. Aligned from the right.

When It Refuses, It Is Helping

THE ERROR PEOPLE FIND INFURIATING, DEFENDED

Six gauges by two hundred and forty samples, and you want to subtract one offset per gauge. You have six offsets. So you write the six by two hundred and forty array minus the array of six, and it raises a ValueError.

This is the error people complain about, and it is the rule protecting you. Align from the right: two hundred and forty against six. Not equal, and neither is one. NumPy has no way to know whether your six numbers were meant to go across the rows or along the samples, and rather than guess it stops.

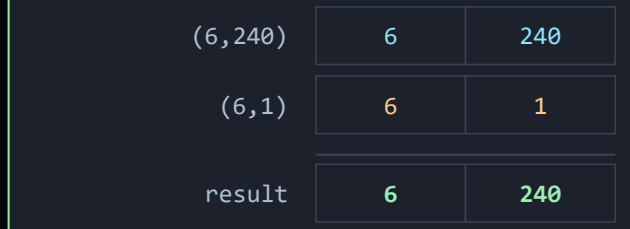
The fix is to say which you meant, by reshaping the offsets to six by one. Then the last axes are two hundred and forty against one, which stretches, and the first are six against six, which match.

Read the message. It names both shapes. Almost every ValueError you will meet in this course is this one, and the two shapes in the message are enough to solve it without looking at anything else.

THE CLASH

240 against 6. Neither is 1. It stops.

SAYING WHICH YOU MEANT



`offsets.reshape(6, 1)` or `offsets[:, None]`

The One That Does Not Raise

THE SINGLE MOST EXPENSIVE MISUNDERSTANDING IN THIS COURSE

One hundred predictions, shape one hundred. One hundred measurements, shape one hundred comma one — because they came out of a network, or out of a column slice, or out of anything that kept its second axis.

Subtract them. Align from the right: one hundred against one, which stretches. Then a missing axis against one hundred, which also stretches. The result has shape one hundred by one hundred.

Nothing raises. You then take the mean of that, get a number, square-root it, and write down an RMS error that is wrong — usually by a factor of a few, always plausibly. It is a ten thousand element matrix where you expected a hundred-element vector, and the only visible symptom is a number you have no independent way of checking.

Two defences. Print both shapes on the line above the subtraction, which takes one second. And when a number surprises you, check the shape of the thing you computed it from before you check anything else.

The exercise notebook makes you produce this bug deliberately and then reports both RMS values side by side. Do that section. It is the twenty minutes this whole lecture is really for.

NO ERROR, AND NOT WHAT YOU WANTED

(100,)	·	100
(100,1)	100	1
result	**100**	**100**

Both stretched. 10,000 elements, silently.

THE ONE-SECOND DEFENCE

```
print(pred.shape, meas.shape)
(100,) (100, 1)

err = pred - meas.ravel()
err.shape                  (100,)
```

Adding and Removing an Axis

FOUR TOOLS, AND WHEN EACH IS THE RIGHT ONE

Once you can read a shape mismatch, fixing it is mechanical. There are four ways to change a shape and each says something slightly different about your intent.

None inside the brackets inserts an axis of length one at that position. This is the one to reach for when you are setting up a broadcast on purpose — it is short, and it puts the new axis exactly where you can see it.

reshape takes the shape you want and is the clearest choice when the target shape is a fact about your data rather than a broadcasting trick. Minus one means work this axis out from the total.

ravel flattens to one dimension, and squeeze removes every axis of length one. Squeeze is convenient and slightly dangerous: it removes axes you did not think about, so on an array whose first dimension happens to be one it will surprise you.

THE FOUR, AND WHAT THEY SAY

x[:, None]	add an axis, for a broadcast
x.reshape(6, 1)	state the shape you want
x.ravel()	flatten to (n,)
x.squeeze()	drop every length-1 axis

```
x.shape            (6,)
x[:, None].shape   (6, 1)   a column
x[None, :].shape   (1, 6)   a row
x.reshape(-1, 1)   (6, 1)   same thing
```

A column is (n, 1). L2.2 makes that the convention for every network input.

When the N by N Is the Point

THE SAME CONSTRUCTION AS SLIDE 11, USED ON PURPOSE

The pairwise distance matrix. Given N positions, produce the N by N array whose entry i, j is the distance between position i and position j.

The naive version is a double loop. The broadcast version is one line: subtract the array as a row from the array as a column, and take the absolute value.

That is exactly the construction that produced the bug two slides ago. Here it is what you wanted, which is the honest summary of broadcasting — it is a sharp tool that does precisely what you asked and has no way of knowing what you meant.

This matters beyond the example. A distance matrix is what a kernel method needs, what a nearest-neighbour search needs, and — in lecture five point one — what an adjacency matrix on a graph looks like.

$$D_{ij} = |x_i - x_j|, \qquad i, j = 0, 1, \ldots, N-1$$

FOUR POSITIONS, SIXTEEN DISTANCES

0	1	2	3
1	0	1	2
2	1	0	1
3	2	1	0

symmetric, zero on the diagonal

ONE LINE

```
d = np.abs(x[:, None] - x[None, :])
d.shape                (4, 4)
# the same trick, wanted this time
```

Reductions, and Which Axis Goes

THE AXIS ARGUMENT NAMES WHAT DISAPPEARS

Sum, mean, max and the rest all take an axis argument, and the rule for reading it is one line: the axis you name is the axis that disappears.

So summing a six by two hundred and forty array along axis nought collapses the six and leaves two hundred and forty — one number per sample, summed over gauges. Along axis one it leaves six — one number per gauge.

People get this backwards constantly, including me, and the reliable fix is not to memorise it but to print the shape of the result and check it against what you meant.

keepdims is the argument that makes the result broadcast back cleanly. With it, the collapsed axis stays as a length of one instead of vanishing, so the mean of a six by two hundred and forty array along axis one has shape six comma one — exactly the shape you need to subtract it from the original.

WHICH AXIS DISAPPEARS


```
a.shape                          (6, 240)
a.sum(axis=0).shape              (240,)
a.sum(axis=1).shape              (6,)
a.mean(axis=1, keepdims=True)    (6, 1)

centred = a - a.mean(axis=1, keepdims=True)
```

That last line is why keepdims exists.

Vectorisation, Measured

THE SAME FORMULA, THREE WAYS, TWO HUNDRED THOUSAND POINTS

The cantilever deflection formula, evaluated at two hundred thousand positions: once as an explicit loop with append, once as a list comprehension, and once as a single NumPy expression.

On a typical Colab CPU the loop takes of the order of a tenth of a second, the comprehension a little less, and the NumPy expression about a millisecond. A speed-up in the tens to low hundreds.

Two things in that table are worth more than the headline. The comprehension is barely faster than the loop, because the arithmetic still happens one Python object at a time — vectorisation is not about syntax. And the gap widens with array size, which is what the figure on the right shows.

The digits depend on the machine and on whether Colab has given you a busy core. The magnitude is the message. If you measure forty and this slide says eighty, nothing is wrong.

$$y(x) = \frac{w\,x^2(6L^2 - 4Lx + x^2)}{24\,E\,I}$$

N = 200,000, FASTEST OF FIVE RUNS

for loop	≈ 100 ms
comprehension	≈ 90 ms — barely better
NumPy	≈ 1 ms
speed-up	tens to low hundreds

AND IT WIDENS WITH SIZE

Schematic — axes are orders of magnitude, and deliberately not calibrated.

And When Not to Bother

THE HONEST LIMITS OF THE ADVICE ON THE LAST SLIDE

Vectorise for speed only when speed is the problem. A clear loop that runs once and takes a tenth of a second is better than a clever expression nobody can read, including you in six weeks.

The interpreter overhead that vectorisation removes is fixed per element. So when the per-element work is expensive — calling a solver, reading a file, evaluating a model — the overhead is negligible and a loop is perfectly reasonable. That is why the training loops in this course are loops.

What you should not do is write the loop when the vectorised form is also the clearer one, which for elementwise arithmetic it almost always is. Subtracting an offset from every reading is one line either way, and the one-line version says what it means.

There is a memory cost too, and it appears exactly where you would expect. A broadcast that produces an N by N array from two N-element arrays allocates N squared numbers. At N of a hundred that is nothing; at N of a hundred thousand it is eighty gigabytes and your notebook dies.

A DECISION, NOT A DOCTRINE

VECTORISE

Elementwise arithmetic over a whole array, run many times, where the expression is also the clearer statement of intent.

LOOP

Expensive work per item — a solver call, a file read, a training epoch. The interpreter overhead disappears into the noise.

EITHER, HONESTLY

Anything that runs once and finishes before you look up. Write whichever you will understand in six weeks.

AND THE MEMORY COST

An N by N broadcast allocates N squared numbers.

Figures an Engineer Can Defend

ONE RULE, AND IT IS NOT NEGOTIABLE

Every axis is labelled, and every label carries its unit. A plot of deflection against position with bare numbers on the axes is not a result — it is a picture of some numbers, and a reader cannot tell whether the answer is millimetres or metres.

That is the only presentation rule this course enforces, and it is enforced because it is the one that separates a figure you can put in a report from one you cannot.

The mechanics are two objects. A figure is the canvas; axes are the coordinate system drawn on it. Make both at once with subplots, then call methods on the axes rather than on the pyplot module — it reads better and it is the only form that works when you have more than one panel.

One practical note about notebooks. Keep one figure in one cell. Jupyter renders a figure when the cell that created it finishes, so a figure built across two cells shows up empty in the first and complete in the second, which is confusing for exactly as long as it takes to learn.

THE SKELETON

```
fig, ax = plt.subplots(figsize=(7.2, 4.4))
ax.plot(x_m, y_mm, label='w = 500 N/m')
ax.set_xlabel('position x  [m]')
ax.set_ylabel('deflection  [mm]')
ax.legend()
ax.grid(True, alpha=0.3)
fig.tight_layout()
```

THE CHECKLIST

Both axes labelled. Both units stated. A legend if there is more than one curve. A grid you can read a value off. And a caption that says what the figure shows rather than repeating its title.

Several Curves, and Telling Them Apart

THREE HABITS THAT COST NOTHING

Distinguish curves by more than colour. Some of your readers print in greyscale and some are colour-blind, so give each curve a line style as well. This costs one argument per call.

Label the curves, not the legend. Put the label on each plot call — label equals w equals two hundred and fifty newtons per metre — and let the legend assemble itself. A hand-written legend list drifts out of order the moment you add a curve, and it drifts silently.

Order the legend the way the curves are ordered on the page, when they have a natural order. A load sweep whose legend runs from the smallest load to the largest is read at a glance; the same sweep in the order you happened to write the loops is not.

None of this is decoration. A figure that takes ten seconds to decode instead of one is a figure that does not get read, and in an exercise report that means the argument you made with it does not land.

A LOAD SWEEP, DONE PROPERLY

125 N/m	solid
250 N/m	dashed
375 N/m	long dash
500 N/m	dotted

FIGURES

When a Linear Axis Lies

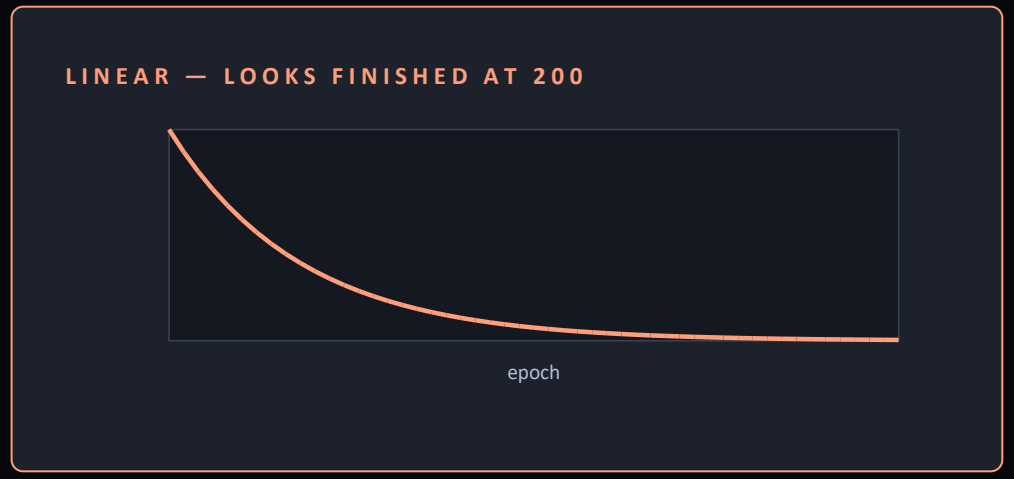
THE ONE AXIS CHANGE YOU WILL MAKE MOST OFTEN

A quantity that falls over several orders of magnitude is unreadable on a linear axis. Everything after the first decade is squashed against zero, and the interesting part of the curve — the part where progress stopped — is invisible.

A logarithmic y-axis fixes it, and one call does it. From lecture two point two onwards every loss curve in this course is plotted on a log axis, and that is not a preference: on a linear axis, a training run that improved by a factor of ten in its last three hundred epochs looks exactly like one that did nothing.

The same argument applies to a residual, to an error against step size, and to anything with the word convergence near it.

One caveat worth stating. A log axis cannot show zero or a negative number, so a quantity that legitimately crosses zero needs a symmetric log axis or a linear one with a sensible range. Do not silently drop the points that will not plot.



The Residual Plot

THE FIGURE THAT ACTUALLY TELLS YOU WHETHER A MODEL FITS

Plot the measurements and the model together and almost any model looks plausible. Plot the difference between them against the input and the truth comes out immediately.

A good fit gives residuals scattered about zero with no pattern — noise, and nothing else. A residual plot with structure in it means the model is missing a term, and the shape of the structure usually tells you which term.

That is why the convention in this course is a two-panel figure: data and fit on top, residuals underneath, sharing an x-axis. You will produce one in Ex01 notebook three and then again in every exercise from Ex03 onwards.

Report the RMS error alongside it, in the units of the measurement. A dimensionless number is not a result — an RMS of nought point nought four millimetres means something, particularly when your dial gauge is only good to nought point nought four.

$$\mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_{i=0}^{N-1}(\hat{y}_i - y_i)^2}$$

TWO PANELS, ONE X-AXIS

No pattern in the lower panel. That is what a good fit looks like.

DEBUGGING

When an Array Expression Misbehaves

SIX STEPS, AND THEY RESOLVE MOST CASES IN UNDER A MINUTE

This extends the recipe from the end of the last lecture with the steps that are specific to arrays. It applies whether the expression raised or — worse — did not.

Print every shape on the line, immediately above the failure, before forming any theory. Then say out loud what shape you expected and why: one residual per collocation point, so the shape should be N. If you cannot state that, you have found the bug.

Feed the two shapes to the alignment table. It tells you which axis clashed, or which axis was stretched when you did not want it stretched, and it is the same table that has been on the right-hand side of the last six slides.

Then look for a stray axis of length one. It came from a network output, from keepdims, or from slicing with nought colon one instead of nought. And if the numbers are wrong rather than the shape, check the dtype before anything else — integer arrays truncate, and mixed float32 and float64 will bite you from L2.2 onwards.

IN ORDER, AND WITHOUT SKIPPING

1	print every shape on the line
2	say what you expected, and why
3	align the two shapes from the right
4	look for a stray axis of length 1
5	check the dtype if the values are wrong
6	shrink it to four elements and check by hand

PAUSE THE VIDEO HERE

Ex01 notebook 02, section 4, turn 1, asks you to predict eight result shapes before running anything. Predict first. Getting one wrong there is worth more than getting all eight right afterwards.

THE EXERCISE

Ex01 Notebooks 02 and 03

NOTEBOOK 02 IS THE ONE TO PROTECT

Notebook 02 is the longest in Ex01 and it says so in its first paragraph. Seven TODO cells: arrays against lists, dtype and shape, indexing, then four separate exercises on broadcasting, then the timed comparison.

Three of those broadcasting exercises deserve naming. One asks you to predict eight result shapes before running anything — predict, then check. One makes you produce the shape-one-hundred against shape-one-hundred-by-one bug deliberately and prints both RMS values side by side. And one builds a distance matrix, where the N by N result is what you wanted.

Notebook 03 is figures: the load sweep, the log axis, and the two-panel residual plot. Two TODO cells and a checklist you should keep.

If you are short of time, do notebook 02 properly and skim notebook 03. The figure conventions can be picked up while doing Ex03; the broadcasting cannot be picked up in week seven, which is when you would otherwise need it.

THE NOTEBOOKS

00	environment check — done last time
01	the language — done last time
02	arrays and BROADCASTING — 7 TODOs
03	engineering figures — 2 TODOs

Both notebooks use the same cantilever as notebook 01, so every shape you handle still means something physical.

PAUSE THE VIDEO HERE, AND FOR LONGER

Ex01 notebooks 02 and 03. Budget ninety minutes for notebook 02 and thirty for notebook 03. It is the best-value ninety minutes in Part 1.

SUMMARY

Key Takeaways

1	**Print the shape, every time**	One line, one second. It converts the commonest class of bug in this course from twenty minutes into ten.
2	**Equal, or one, from the right**	The whole broadcasting rule, with no second clause. Write the two shapes above each other and check the columns.
3	**A ValueError is the rule helping**	It means NumPy could not tell what you meant. Read the message: it names both shapes, which is enough to fix it.
4	**(N,) against (N, 1) does not raise**	It gives you an N by N matrix and a plausible wrong number. This is the one that costs an afternoon.
5	**Every axis labelled, with units**	The only presentation rule enforced in this course, and the one that separates a result from a picture of numbers.

Next — L2.1: pandas, file I/O, an ODE solved numerically, and the curve fit that deep learning will later have to beat.

SOURCES

References and Further Reading

TWO PIECES OF DOCUMENTATION, AND NOTHING ELSE

The NumPy documentation has a page called Broadcasting which states the rule in the same two clauses I used, with a table of examples. It is about a thousand words and it is the single best thing to read after this lecture.

The NumPy absolute beginners guide covers everything on slides three to seven. The matplotlib usage guide covers slides seventeen to twenty, and its Anatomy of a Figure diagram is worth printing.

Neither of the course textbooks helps here. Prince assumes NumPy throughout and never introduces it, and Goodfellow's chapter 2 is linear algebra rather than the library.

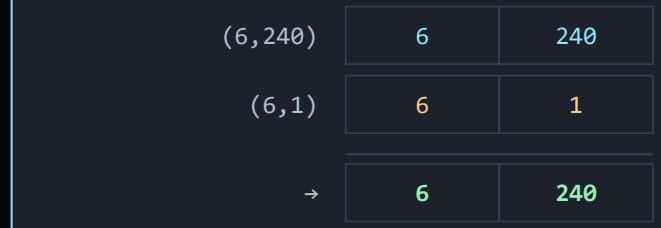
IF YOU READ ONE THING

The NumPy Broadcasting page. Read it after you have done notebook 02 rather than before — the examples land differently once you have made the mistake yourself.

And keep the figure checklist from notebook 03 somewhere you will see it. It is six lines and it will improve every report you write this semester.

IN THIS ORDER

today	Ex01 notebook 02, all seven TODOs
then	the NumPy Broadcasting page
then	Ex01 notebook 03
before L2.2	re-read your own §4 answers
not needed	either course textbook



The one picture to leave the lecture with.

NEXT

L2.1 — Scientific Python

Arrays are the object. Next time they carry real measurements: pandas for tabular data, a CSV that is genuinely messy, an ODE solved numerically — and a two-parameter curve fit that a network will later struggle to beat.

BEFORE THE NEXT RECORDING

work	Ex01 notebook 02 — ninety minutes
work	Ex01 notebook 03 — thirty minutes
read	the NumPy Broadcasting page
keep	the figure checklist

Notebook 02 section 4 is the part that matters. If you do nothing else before the next recording, predict those eight shapes and find out which ones you got wrong.